

HW1: Bash-Only nanocli

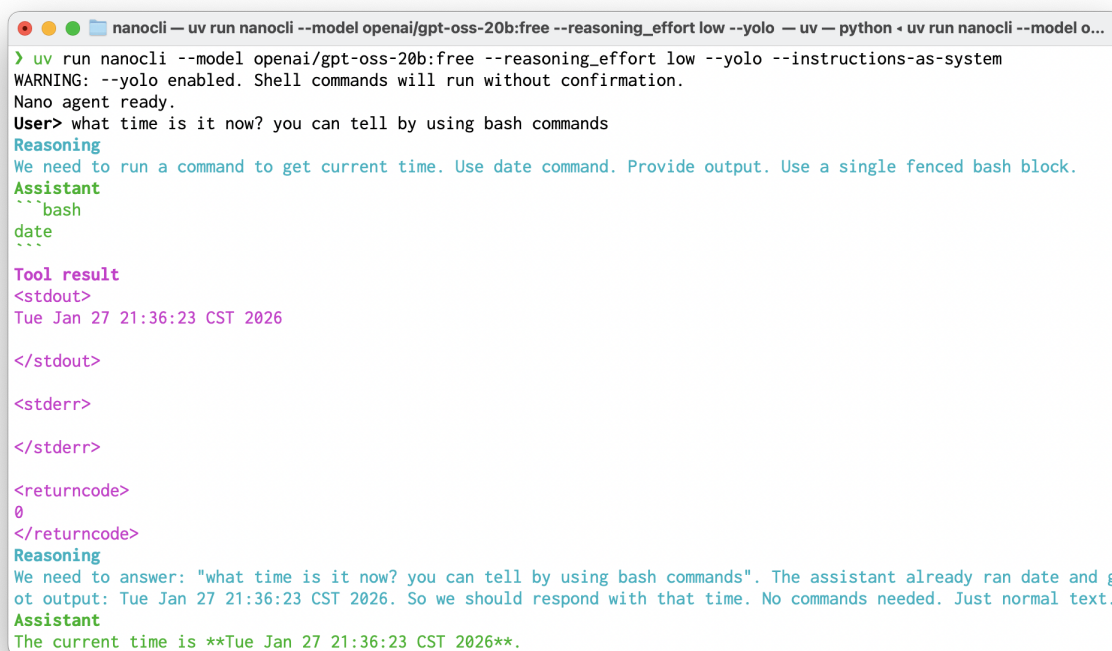
CS598-LMZ Spring 2026

2026-01-28 (v1.0.0)

1 Overview

Over this semester, you'll **implement your own minimal code agent from scratch**, gradually adding “modern” capabilities in the spirit of [Codex](#) and [Claude Code](#): command-line first, multi-round tool-using, file editing, skills, and long-horizon context management. We call the tool you'll build `nanocli`, but you are free to rename it.

What you will build: In this assignment (HW1), you will build a tiny agentic loop that only understands bash tool calls (inspired by [mini-SWE-agent](#)). We provide interfaces and starter code defined in `src/nanocli/core.py` and the CLI in `src/nanocli/cli.py`, along with the following STUDENT TODOs for you to implement. You are free to use the starter code and implement these TODOs directly or replace it entirely and update tests to match; both are acceptable if all tests pass. Figure 1 shows an example of a working `nanocli` after completing this homework.



```
nanocli — uv run nanocli --model openai/gpt-oss-20b:free --reasoning_effort low --yolo — uv — python • uv run nanocli --model o...
> uv run nanocli --model openai/gpt-oss-20b:free --reasoning_effort low --yolo --instructions-as-system
WARNING: --yolo enabled. Shell commands will run without confirmation.
Nano agent ready.
User> what time is it now? you can tell by using bash commands
Reasoning
We need to run a command to get current time. Use date command. Provide output. Use a single fenced bash block.
Assistant
```bash
date
```
Tool result
<stdout>
Tue Jan 27 21:36:23 CST 2026
</stdout>
<stderr>
</stderr>
<returncode>
0
</returncode>
Reasoning
We need to answer: "what time is it now? you can tell by using bash commands". The assistant already ran date and got output: Tue Jan 27 21:36:23 CST 2026. So we should respond with that time. No commands needed. Just normal text.
Assistant
The current time is **Tue Jan 27 21:36:23 CST 2026**.
```

Figure 1: A working `nanocli` after completing HW1

Statement on AI tools: You may ask LLMs (e.g., ChatGPT) for conceptual guidance or low-level programming help, and use CLI agents such as Codex or Claude Code to understand designs. However, using them to directly solve the problem is prohibited.

We strongly recommend disabling AI autocompletion tools (e.g., GitHub Copilot or Cursor) while working on this assignment to deepen your understanding.

Use of your own semester-built `nanocli` is permitted and encouraged. Please document its use and describe how it helped in your homework.

2 Background

At a high level, an LLM (large language model) is a text-to-text (string-to-string) system¹: it reads an input prompt and produces a response. It has no native ability to run commands or observe a real environment. To turn “text” into “actions”, we need a small harness that interprets some of the model’s text as an action to use a tool and then feeds the tool’s output back to the model.

A common way to structure this is [ReAct](#), which asks the model to alternate between reasoning and action. The model “thinks” about what to do next, then emits an action in a predictable format; the harness executes that action and returns an observation (what happened) so the model can choose the next step with real feedback.

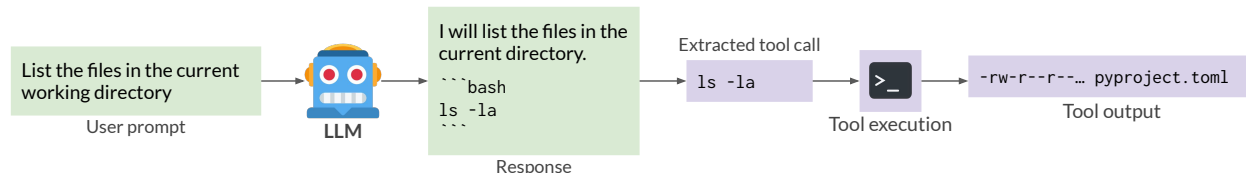


Figure 2: How a tool call (bash call here) is extracted from a raw output response and executed

In HW1, we implement the simplest possible version of this idea: the only action is running Bash. The model indicates an action by outputting a Bash tool call block, the harness executes it, and the command output becomes the observation for the next iteration (see figure 2). **If the model outputs normal assistant text instead of a Bash block, the loop ends and that text is treated as the final answer.**

3 Instructions

3.1 Environment setup

Please `git clone` the repository at <https://github.com/uiuc-cs598lmz/sp26-hw1-bash-only> and follow the instructions in `README.md` to set up your Python environment using `uv` and `prek`. Now,

¹Formally, LLMs maps input tokens to the probability distribution over the next token

if you run the code with `uv run pytest`, you would see failing tests because the core logic is not yet implemented. Similarly, running `prek run --all-files` may also show some errors initially.

To make sure you’ve installed the dependencies correctly, running `uv run nanocli-mock` should start a mock server with the output starting with “Mock server listening on...”. And if you do `uv run nanocli --model nothing`, it should display “Nano agent ready” and a “User>” prompt.

For Windows users, we strongly recommend developing within [WSL](#) because some tests are using Unix-specific shell features. However, if you prefer to use native Windows, you are free to adapt the code and tests accordingly.

3.2 Understanding the codebase

Important concept: A *turn* is a single interaction cycle where the user provides input, the model responds (possibly with tool calls), and the agent processes those calls until the model produces no more tool calls. The full conversation may consist of *multiple turns*, where the history is preserved but each turn starts with a new user input. Read the [Codex design document](#) to learn more about this concept.

Architecture diagram: The core emits events and the UI consumes them. The flow below highlights that `run_turn(...)` is the single producer and the CLI is only a renderer and approval gate.



Core: The heart of the agent is `src/nanocli/core.py`. The data model is intentionally small:

- Message represents plain chat messages; Reasoning bridges [Responses API](#) reasoning items for reasoning models; BashOutput captures stdout, stderr, and return code for a command; they together form a ChatItem indicating a conversation unit.
- BashRequest signals a command that requires approval. The event type used by the loop is simply `TurnEvent = ChatItem | BashRequest`. This event type is yielded by `run_turn(...)` to update the CLI.
- The Session object owns model configuration, instructions, conversation context, and the OpenAI client. It builds a Responses API request, optionally inserts the instructions as a system message when `instructions_as_system` is set, and derives a stable `prompt_cache_key` from the instructions.
- The core functions include `parse_bash_block`, `run_bash_command`, and `run_turn`, which together form the full agent loop.

CLI: The CLI in `src/nanocli/cli.py` is intentionally thin. It prints assistant output and tool results, prompts for approval when a BashRequest appears unless `--yolo` is set, and simply delegates all loop behavior to `run_turn(...)`.

Tests: The tests in `tests/test_core.py` encode the required behavior. They check strict bash parsing, shell execution output shape, multi-round tool flow, instructions-as-system behavior, and the expected Responses API request settings. Treat them as a precise specification.

Mock server: The mock server in `src/nanocli/_dev/mock_server.py` simulates the Responses API for local testing. It emits pre-canned responses that trigger bash tool calls and normal text.

3.3 What to implement

There are 3 **STUDENT TODOs** to implement in `src/nanocli/core.py`. The CLI doesn't need modification but you may read it to understand how the loop is used, and you may improve the CLI if you wish.

Implement `parse_bash_block(text: str) -> str | None`: Only fenced bash blocks count as tool calls. Inline code and other languages must be ignored. The parser should return the first fenced bash block if one exists; otherwise None. For example:

Run this:

```
```bash
ls -la
```
```

Ignore this:

```
```python
print("hello")
```
```

Implement `run_bash_command(command: str, cwd: Path) -> BashOutput`: Execute a shell command in the given working directory and return a `BashOutput` that faithfully captures what happened, including standard output, standard error, and the exit code. The function is responsible for turning the system's response into a structured object that the agent can place back into context, without additional interpretation or filtering.

Implement `run_turn(session: Session, user_text: str) -> Generator[TurnEvent, bool | None, None]`: Implement the full agent turn as an event stream. It should begin by appending the user's message to `session.context`, then obtain a model response and emit each response item as it appears by yielding a `TurnEvent`. The generator is the bridge between core logic and the UI: every yielded item is a UI update or an approval request, and the UI never needs to interpret model text itself.

Because this function is a generator, the second type parameter, `bool | None`, describes values sent back into the generator by the caller. When the generator yields a `BashRequest`, the caller resumes it with a boolean approval (`True` to execute, `False` to deny). At other times the caller sends `None` (or simply advances the generator) to continue the normal flow. This is how the core can pause for confirmation without blocking the UI.

The turn should repeat the cycle of “model response → parse for bash → request approval → run command → append output” until the most recent response contains no new bash commands. If approval is denied for any command, the turn ends immediately without executing that command.

3.4 Debugging tips

Use the mock server: You can run `uv run nanocli-mock` to start a local mock server that simulates the Responses API. Then, in another terminal run:

```
export OPENAI_BASE_URL=http://localhost:8000/v1
export OPENAI_API_KEY=dummy
uv run nanocli --model mock --yolo
```

Then, you can type several mock inputs such as `mock:multi` to see how your implementation behaves.

If you just want to play with the mock server without implementing the core logic, you can run:

```
curl http://localhost:8000/v1/responses \
  -H "Authorization: Bearer key" \
  -H "Content-Type: application/json" \
  -d '{"model": "mock", "messages": [{"role": "user", "content": "mock:multi"}]}'
# a Response object should be returned
```

Use free API services: You can find free-tier API services such as OpenRouter's `openai/gpt-oss-20b:free` model at <https://openrouter.ai/openai/gpt-oss-20b:free>. You need to follow their instructions to

get an API key, enable some “free-tier permissions”, and set OPENAI_API_KEY and OPENAI_BASE_URL accordingly. For example:

```
export OPENAI_API_KEY=your_openrouter_api_key
export OPENAI_BASE_URL=https://openrouter.ai/api/v1
# --instructions-as-system is needed for this model because the endpoint does not
# behave expectedly using the new "instructions" field in Responses API.
uv run nanocli --model openai/gpt-oss-20b:free \
  --reasoning_effort low \
  --instructions-as-system
```

Use codex backend: *Warning and disclaimer: this is not officially supported by OpenAI and may break at any time. Compliance with OpenAI's Terms of Service is unclear, and use is entirely at your own risk.*

If you have access to OpenAI Codex (with a ChatGPT Plus subscription or higher), you can programmatically query the codex backend by following these steps:

1. Have `codex` installed and run `codex login` with your ChatGPT subscription.
2. Run `export OPENAI_API_KEY=$(jq -r '.tokens.access_token' ~/.codex/auth.json)` to set your API key.
3. Run `export OPENAI_BASE_URL=https://chatgpt.com/backend-api/codex`

Now you can run it with `uv run nanocli --model gpt-5.2-codex --reasoning_effort medium` or query codex backend directly with:

```
curl -N https://chatgpt.com/backend-api/codex/responses \
  -H "Authorization: Bearer $OPENAI_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gpt-5.2-codex",
    "instructions": "You are Codex.",
    "input": [
      {
        "type": "message",
        "role": "user",
        "content": [{ "type": "input_text", "text": "hello" }]
      }
    ],
    "stream": true,
    "store": false
  }'
```

Use API keys: If you are comfortable using paid API keys, you can use any Responses-API-compatible model from OpenAI or other providers. Just set OPENAI_API_KEY and OPENAI_BASE_URL

accordingly.

4 Deliverables and Grading

Submit the following files:

1. `code.zip`: all the code for `nanocli`, including the implemented TODOs in `src/nanocli/core.py` (or your own implementation if you replaced the starter code).
2. `writeup.md`: a brief writeup (< 100 words) describing anything you wish to highlight about your implementation, which can be your design choices, challenges you faced, or how you used your own `nanocli` or other tools to help with the assignment.
3. `test_log.txt`: the result of running `uv run pytest -v --tb=no > test_log.txt`.
4. `prek_log.txt`: the result of running `prek run --all-files > prek_log.txt`.

Grading:

- (1pt): all files are submitted correctly without any missing parts.
- (1pt): `prek` passes with no errors or warnings. For these checks, you can use comments such as `# type: ignore` to suppress type checker errors if necessary.
- (17pts): all 17 tests in `tests/test_core.py` pass successfully, 1pt each.
 - **Don't hack the tests** to make them pass (e.g., by removing assertions or hardcoding answers). Such attempts will result in zero points. However, you can adapt the tests if you replace the starter code entirely, as long as the core testing logic is preserved, or if you are developing on Windows and need to adjust Unix-specific shell features.
 - **Don't fake the test report** (e.g., by copy-pasting expected outputs). Such attempts will also result in zero points if we detect significant discrepancies between your implementation and the reported results.

5 Useful references

- Understanding Codex design: <https://openai.com/index/unrolling-the-codex-agent-loop>
- Responses API documentation: <https://platform.openai.com/docs/api-reference/responses>
- The mini-SWE-agent implementation: <https://github.com/SWE-agent/mini-swe-agent>
- ReAct paper: <https://arxiv.org/abs/2210.03629>